

Contents

1	前置作業	
1.1	浮點數	
1.2	__int128_t	
2	資料結構	
2.1	BIT	
2.2	線段樹	
2.3	吉司機線段樹	
3	字串	
3.1	字典樹 trie	
3.2	LCP	
3.3	KMP	
4	最短路	
4.1	dijkstra	
4.2	SPFA	
5	圖論	
5.1	建點	
5.2	點直線、直線直線關係	
5.3	凸包問題	
5.4	旋轉卡尺	
5.5	最大流	
5.6	最小割	
6	數論	
6.1	快速冪	
6.2	取餘數	
6.3	判斷質數	
6.4	找一質因數	
6.5	找全質因數	
7	數學	
7.1	定理	

1 前置作業

1.1 浮點數

```

1 #define eps 1e-6
2 int sgn(double x){return (x>eps)-(x<-eps);}
3 // return (-1,0,1)->(負數,0,正數)
4
5 /*
6  !! 浮點數縮放記得 round() !!
7  round(x*10000.0);
8  */

```

1.2 __int128_t

```

1 void print128(__int128_t x){
2     if(x==0){
3         cout<<0;
4         return;
5     }
6     if(x<0){
7         cout<<"-";
8         x=-x;
9     }
10    string s;
11    while(x){
12        s+=char('0'+x%10);
13        x/=10;
14    }
15    reverse(s.begin(),s.end());
16    cout<<s;
17 }

```

2 資料結構

2.1 BIT

```

1 struct Binary_Indexed_Tree{//一定要用 1-base
2     int n;
3     vector<int> bit;
4     void init(int _n){
5         n=_n+1;

```

```

6         bit=vector<int>(n,0);
7     }
8     int lowbit(int x){return x&-x;}
9     void update(int x,int v){
10         for(;x<n;x+=lowbit(x)) bit[x]+=v;
11     }
12     void range_update(int l,int r,int v){
13         update(l,v);
14         update(r+1,-v);
15     }
16     int qry(int x){
17         int ans=0;
18         for(;x>0;x-=lowbit(x)) ans+=bit[x];
19         return ans;
20     }
21 }

```

2.2 線段樹

```

1 struct Segtree{ // 0-base
2     int n;
3     vector<int> seg;
4     vector<int> tag;
5     vector<int> *a;
6     #define cl(x) (x<<1)+1
7     #define cr(x) (x<<1)+2
8     #define mid ((l+r)>>1)
9     void init(vector<int> &x){
10         n=x.size();
11         seg=vector<int> (n*4+5);
12         tag=vector<int> (n*4+5);
13         a=&x;
14     }
15     void pull(int id){
16         seg[id]=seg[cl(id)]+seg[cr(id)];
17     }
18     void build(int l,int r,int id=0){
19         if(l==r){
20             seg[id]=(*a)[l];
21             return;
22         }
23         build(l,mid,cl(id));
24         build(mid+1,r,cr(id));
25         pull(id);
26     }
27     // <單點更新>
28     void update(int l,int r,int x,int v,int id=0){
29         if(l==r){
30             seg[id]=v;
31             return;
32         }
33         if(x<=mid) update(l,mid,x,v,cl(id));
34         if(mid<x) update(mid+1,r,x,v,cr(id));
35         pull(id);
36     }
37     // </單點更新>
38     // <區間更新>
39     void add(int id,int l,int r,int v){
40         seg[id]+=v*(r-l+1);
41         tag[id]+=v;
42     }
43     void push(int id,int l,int r){
44         if(l==r || tag[id]==0) return;
45         add(cl(id),l,mid,tag[id]);
46         add(cr(id),mid+1,r,tag[id]);
47         tag[id]=0;

```

```

48 }
49 void update(int l,int r,int sl,int sr,int v,int
    id=0){
50     if(sl<=l && r<=sr){
51         add(id,l,r,v);
52         return;
53     }
54     push(id,l,r);
55     if(sl<=mid) update(l,mid,sl,sr,v,cl(id));
56     if(mid<sr) update(mid+1,r,sl,sr,v,cr(id));
57     pull(id);
58 }
59 //<區間更新>
60 int qry(int l,int r,int sl,int sr,int id=0){
61     if(sl<=l && r<=sr) return seg[id];
62     push(id,l,r); //區間更新用
63     int ans=0;
64     if(sl<=mid) ans+=qry(l,mid,sl,sr,cl(id));
65     if(mid<sr) ans+=qry(mid+1,r,sl,sr,cr(id));
66     return ans;
67 }

```

2.3 吉司機線段樹

```

1 struct SegTreeBeat{
2     struct node{int mx,smx,cntmx,sum;};
3     #define cl(x) (x<<1)+1
4     #define cr(x) (x<<1)+2
5     #define mid ((l+r)>>1)
6     vector<node> seg;
7     vector<int> tag;
8     vector<int> *a;
9     int n;
10    void pull(int id){
11        seg[id].sum=seg[cl(id)].sum+seg[cr(id)].sum;
12
13    if(seg[cl(id)].mx==seg[cr(id)].mx){
14        seg[id].mx=seg[cl(id)].mx;
15        seg[id].smx=max(seg[cl(id)].smx,seg[cr(id)].smx);
16        seg[id].cntmx=seg[cl(id)].cntmx+seg[cr(id)].cntmx;
17    }else if(seg[cl(id)].mx>seg[cr(id)].mx){
18        seg[id].mx=seg[cl(id)].mx;
19        seg[id].smx=max(seg[cl(id)].smx,seg[cr(id)].mx);
20        seg[id].cntmx=seg[cl(id)].cntmx;
21    }else{
22        seg[id].mx=seg[cr(id)].mx;
23        seg[id].smx=max(seg[cr(id)].smx,seg[cl(id)].mx);
24        seg[id].cntmx=seg[cr(id)].cntmx;
25    }
26
27    void init(vector<int> &v){
28        n=v.size();
29        seg.resize(n*4+5);
30        tag=vector<int>(n*4+5,0);
31        a=&v;
32    }
33    void build(int l,int r,int id=0){
34        if(l==r){
35            seg[id]={(*a)[l],LLONG_MIN,1,(*a)[l]};
36            return;
37        }
38        build(l,mid,cl(id));
39        build(mid+1,r,cr(id));
40        pull(id);
41    }
42    void apply_chmin(int id,int v){

```

```

43     if(seg[id].mx<=v) return;
44     seg[id].sum-=(seg[id].mx-v)*seg[id].cntmx;
45     seg[id].mx=v;
46 }
47 void apply_add(int id,int l,int r,int v){
48     seg[id].sum+=v*(r-l+1);
49     if(seg[id].smx!=LLONG_MIN) seg[id].smx+=v;
50     seg[id].mx+=v;
51     tag[id]+=v;
52 }
53 void push(int id,int l,int r){
54     if(l==r) return;
55     if(tag[id]){
56         apply_add(cl(id),l,mid,tag[id]);
57         apply_add(cr(id),mid+1,r,tag[id]);
58         tag[id]=0;
59     }
60     apply_chmin(cl(id),seg[id].mx);
61     apply_chmin(cr(id),seg[id].mx);
62 }
63 void add(int l,int r,int sl,int sr,int v,int id=0){
64     if(sl<=l && r<=sr){
65         apply_add(id,l,r,v);
66         return;
67     }
68     push(id,l,r);
69     if(sl<=mid) add(l,mid,sl,sr,v,cl(id));
70     if(mid<sr) add(mid+1,r,sl,sr,v,cr(id));
71     pull(id);
72 }
73 void chmin(int l,int r,int sl,int sr,int v,int id=0){
74     if(seg[id].mx<=v) return;
75     if(sl<=l && r<=sr && seg[id].smx<v){
76         apply_chmin(id,v);
77         return;
78     }
79     push(id,l,r);
80     if(sl<=mid) chmin(l,mid,sl,sr,v,cl(id));
81     if(mid<sr) chmin(mid+1,r,sl,sr,v,cr(id));
82     pull(id);
83 }
84 int qry(int l,int r,int sl,int sr,int id=0){
85     if(sl<=l && r<=sr) return seg[id].sum;
86     push(id,l,r);
87     int ans=0;
88     if(sl<=mid) ans+=qry(l,mid,sl,sr,cl(id));
89     if(mid<sr) ans+=qry(mid+1,r,sl,sr,cr(id));
90     return ans;
91 }

```

3 字串

3.1 字典樹 trie

```

1 struct trie{
2     trie *nxt[26]={};
3     int cnt=0;
4     int sz=0;
5 };
6 trie *root=new trie();
7 void insert(const string &s){
8     trie *now=root;
9     for(auto i:s){
10        if(now->nxt[i-'a']==NULL){
11            now->nxt[i-'a']=new trie();
12        }

```

```

13     now=now->nxt[i-'a'];
14 }}
15 int qry(string &s){
16     trie *now=root;
17     for(auto i:s){
18         if(now->nxt[i-'a']==NULL) return;
19         now=now->nxt[i-'a'];
20     }
21     return now->cnt;
22 }

```

3.2 LCP

```

1 int lcp(const string &pa,const string &pb){//字串先排序
2     int i=0;
3     while(1<pa.size() && i<pb.size() && pa[i]==pb[i])
4         i++;
5     return i;

```

3.3 KMP

```

1 vector<int> kmp(string s,string t){
2     if(t.size()>s.size()) return {};
3     vector<int> pi(t.size(),0);
4     for(int i=1,j=0;i<t.size();i++){
5         while(j>0 && t[j]!=t[i]){
6             j=pi[j-1];
7         }
8         if(t[j]==t[i]) j++;
9         pi[i]=j;
10    }
11    vector<int> ans;
12    for(int i=0,j=0;i<s.size();i++){
13        while(j>0 && t[j]!=s[i]){
14            j=pi[j-1];
15        }
16        if(t[j]==s[i]) j++;
17
18        if(j==t.size()){
19            ans.push_back(i-j+1);
20            j=pi[j-1];
21        }
22    }
23    return ans;

```

4 最短路

4.1 dijkstra

```

1 struct dijkstra{
2     #define pii pair<int,int>
3     int n,source;
4     vector<vector<pii>> E;
5     vector<int> pre,vis,dis;
6     void init(int _n){
7         n=_n+1;
8         E.assign(n,{});
9     }
10    void add_edge(int u,int v,int w){
11        E[u].push_back({v,w});
12    }
13    void dist(int s){
14        source=s;
15        pre.assign(n,-1);
16        vis.assign(n,0);
17        dis.assign(n,inf);
18        priority_queue<pii,vector<pii>,greater<pii>> pq;

```

```

19        dis[s]=0;
20        pq.push({dis[s],s});
21        while(!pq.empty()){
22            int u=pq.top().second;pq.pop();
23            if(vis[u]) continue;
24            vis[u]=1;
25            for(auto [v,w]:E[u]){
26                if(dis[u]+w<dis[v]){
27                    dis[v]=dis[u]+w;
28                    pq.push({dis[v],v});
29                    pre[v]=u;
30                }
31            }
32            vector<int> get_path(int t){
33                if(dis[t]==inf) return {};
34                vector<int> res;
35                for(int i=t;i!=-1;i=pre[i]) res.push_back(i);
36                reverse(res.begin(),res.end());
37                if(res.empty() || res[0]!=source) return {};
38                return res;
39            }
40        }
41    }

```

4.2 SPFA

```

1 struct spfa{
2     struct edge{int v,w;};
3     int n,source;
4     vector<vector<edge>> E;
5     vector<int> dis,cnt,inq,pre;
6     void init(int _n){
7         n=_n+1;
8         E.assign(n,{});
9     }
10    void add_edge(int u,int v,int w){
11        E[u].push_back({v,w});
12    }
13    bool dist(int s){
14        source=s;
15        dis.assign(n,inf);
16        cnt.assign(n,0);
17        inq.assign(n,0);
18        pre.assign(n,-1);
19        queue<int> q;
20        dis[s]=0;
21        q.push(s);
22        inq[s]=1;
23        while(!q.empty()){
24            int u=q.front();q.pop();
25            inq[u]=0;
26            for(auto [v,w]:E[u]){
27                if(dis[u]!=inf && dis[u]+w<dis[v]){
28                    dis[v]=dis[u]+w;
29                    pre[v]=u;
30                    cnt[v]=cnt[u]+1;
31                    if(cnt[v]>=n-1) return 0;
32                    if(!inq[v]){inq[v]=1;q.push(v);}
33                }
34            }
35            return 1;
36        }
37        vector<int> get_path(int t){
38            if(dis[t]==inf) return {};
39            vector<int> res;
40            for(int i=t;i!=-1;i=pre[i]) res.push_back(i);
41            reverse(res.begin(),res.end());
42            if(res.empty() || res[0]!=source) return {};
43            return res;
44        }

```

5 圖論

5.1 建點

```

1 template<class T>
2 struct pt{
3     T x,y;
4     pt(T _x,T _y):x(_x),y(_y){};
5     pt():x(0),y(0){};
6
7     pt operator+(pt a){return pt(x+a.x,y+a.y);}
8     pt operator-(pt a){return pt(x-a.x,y-a.y);}
9     pt operator*(T c){return pt(x*c,y*c);}
10    pt operator/(T c){return pt(x/c,y/c);}
11
12    T operator*(pt a){return x*a.x+y*a.y;}
13    T operator^(pt a){return x*a.y-y*a.x;}
14
15    bool operator<(const pt &a) const{
16        return x<a.x || (x==a.x && y<a.y);
17    }
18 };
19 using Pt=pt<int>;

```

5.2 點直線、直線直線關係

```

1 auto cross(Pt a,Pt b,Pt c){return (b-a)^(c-a);}
2 auto dot(Pt a,Pt b,Pt c){return (b-a)*(c-a);}
3 double dis(Pt p){return sqrt(p*p);}
4 struct Line{Pt a,b};
5 bool PtOnSeg(Pt p,Line l){ //判斷在線段上
6     return sgn(cross(l.a,l.b,p))==0 &&
7            sgn(dot(p,l.a,l.b))<=0;
8 }
9 double PtSegDist(Pt p,Line l){ //求點線距
10    double ans=min(abs(p-l.a),abs(p-l.b));
11    if(sgn(dot(l.a,l.b,p))<0) return ans;
12    if(sgn(dot(l.b,l.a,p))<0) return ans;
13    return min(ans,
14               abs(cross(p,l.a,l.b))/abs(l.a-l.b));
15 }
16 int PtSide(Pt p,Line l){
17     return sgn(cross(l.a,l.b,p));
18 }
19 bool isInter(Line l1,Line l2){ //線段相交
20     if(PtOnSeg(l2.a,l1) || PtOnSeg(l2.b,l1) ||
21        PtOnSeg(l1.a,l2) || PtOnSeg(l1.b,l2))
22         return 1;
23     return PtSide(l2.a,l1)*PtSide(l2.b,l1)<0 &&
24            PtSide(l1.a,l2)*PtSide(l1.b,l2)<0;
25 }
26 int inPoly(Pt p,vector<Pt> &vec){ //在多邊形中
27     int n=vec.size();
28     int cnt=0;
29
30     for(int i=0;i<n;i++){
31         Pt a=vec[i];
32         Pt b=vec[(i+1)%n];
33
34         if(PtOnSeg(p,{a,b})) return 1;
35
36         if((sgn(a.y-p.y)==1)^(sgn(b.y-p.y)==1)){
37             cnt+=sgn(cross(a,b,p));
38         }
39     }
40 }

```

```

41     return (cnt==0)?0:2;
42 } // (0,1,2) -> (外,線,內)

```

5.3 凸包問題

```

1 struct ConvexHull{
2     vector<Pt> vec;
3     int n;
4     void init(vector<Pt> &v){
5         vec=v;
6         n=v.size();
7     }
8     int cross(Pt o,Pt a,Pt b){return (a-o)^(b-o);}
9     vector<int> point;
10    vector<int> area;
11    vector<Pt> up,down;
12    void build(){
13        up.clear();down.clear();
14        point=vector<int>(n+1);
15        area=vector<int>(n+1);
16        int upsum=0;
17        int downsum=0;
18        for(int i=0;i<n;i++){
19            while(up.size()>=2 &&
20                  cross(up[up.size()-1],
21                        up[up.size()-2],vec[i])<=0){
22                upsum-=cross({0,0},up[up.size()-2],
23                             up[up.size()-1]);
24                up.pop_back();
25            }
26            up.push_back(vec[i]);
27            if(up.size()>1)
28                upsum+=cross({0,0},up[up.size()-2],
29                             up[up.size()-1]);
30
31            while(down.size()>=2 &&
32                  cross(down[down.size()-1],
33                        down[down.size()-2],vec[i])>=0){
34                downsum-=cross({0,0},down[down.size()-2],
35                               down[down.size()-1]);
36                down.pop_back();
37            }
38            down.push_back(vec[i]);
39            if(down.size()>1)
40                downsum+=cross({0,0},down[down.size()-2],
41                               down[down.size()-1]);
42
43            area[i+1]=abs(upsum-downsum);
44
45            if(i+1==1) point[i+1]=1;
46            else point[i+1]=up.size()+down.size()-2;
47        }
48        double getarea(int p){
49            return area[p]/2.0;
50        }
51        int getnum(int p){
52            return point[p];
53        }
54        vector<Pt> getall(){
55            vector<Pt> ans;
56            for(int i=0;i<down.size();i++)
57                ans.push_back(down[i]);
58            for(int i=up.size()-2;i>=1;i--)
59                ans.push_back(up[i]);
60            return ans;
61        }

```

62 };

5.4 旋轉卡尺

```

1 double RotatingCalipers(vector<Pt> &v){ //逆時針
2     int n=v.size();
3     double dist=0;
4     if(n==1) return 0;
5     if(n==2) return dis(v[0]-v[1]);
6     for(int i=0,j=2;i<n;i++){
7         Pt a=v[i],b=v[(i+1)%n];
8         while(cross(v[j],a,b)<cross(v[(j+1)%n],a,b))
9             j=(j+1)%n;
10        dist=max(dist,dis(v[j]-a));
11        dist=max(dist,dis(v[j]-b));
12    }
13    return dist;
14 }
```

5.5 最大流

```

1 struct Dinic{
2     struct edge{int v,f,re;};
3     int n,s,t;
4     vector<vector<edge>> E;
5     vector<int> level;
6     void init(int _n,int _s,int _t){
7         n=_n;s=_s;t=_t;
8         E.assign(n,{});
9     }
10    void add_edge(int u,int v,int w){
11        int Su=E[u].size();
12        int Sv=E[v].size();
13        E[u].push_back({v,w,Sv});
14        E[v].push_back({u,0,Su});
15    }
16    int bfs(){
17        level.assign(n,-1);
18        queue<int> q;
19        level[s]=0;
20        q.push(s);
21        while(!q.empty()){
22            int u=q.front();q.pop();
23            for(auto [v,f,re]:E[u]){
24                if(f>0 && level[v]==-1){
25                    level[v]=level[u]+1;
26                    q.push(v);
27                }
28            }
29            return level[t]!=-1;
30        }
31        int dfs(int u,int nf){
32            if(u==t) return nf;
33            int res=0;
34            for(auto &[v,f,re]:E[u]){
35                if(f>0 && level[v]==level[u]+1){
36                    int tf=dfs(v,min(nf,f));
37                    res+=tf;
38                    nf-=tf;
39                    f-=tf;
40                    E[v][re].f+=tf;
41                    if(nf==0) return res;
42                }
43            }
44            if(!res) level[u]=-1;
45            return res;
46 }
```

```

47     while(bfs()) res+=dfs(s,inf);
48     return res;
49 };
```

5.6 最小割

```

1 struct node{int u,v,id;};
2 vector<node> edge;
3 for(int i=0;i<m;i++){
4     cin>>u>>v>>w;
5     flow.add_edge(u,v,w);
6     edge.push_back({u,v,i});
7 }
8 flow.flow();
9 for(auto &[u,v,id]:edge){
10     if(flow.level[u]!=-1 && flow.level[v]==-1){
11         cout<<id<<" ";
12     }
13 }
```

6 數論

6.1 快速幂

```

1 int ksm(int a,int b,int mod){
2     int ans=1;
3     while(b){
4         if(b&1) ans=mul(ans,a,mod);
5         a=mul(a,a,mod);
6         b>>=1;
7     }
8     return ans;
9 }
```

6.2 取餘數

```

1 int mul(int x,int y,int mod){
2     return (__int128)x*y%mod;
3 }
```

6.3 判斷質數

```

1 // n < 4,759,123,141      3 : 2, 7, 61
2 // n < 1,122,004,669,633  4 : 2, 13, 23, 1662803
3 // n < 3,474,749,660,383  6 : pirms <= 13
4 // n < 2^64              7 :
5 // 2, 325, 9375, 28178, 450775, 9780504, 1795265022
6 // Make sure testing integer is in range [2, n-2] if
7 // you want to use magic.
8 int magic[]={}
9 bool witness(int a,int n,int u,int t){
10     if(!a) return 0;
11     int x=ksm(a,u,n);
12     for(int i=0;i<t;i++){
13         int nx=mul(x,x,n);
14         if(nx==1 && x!=1 && x!=n-1) return 1;
15         x=nx;
16     }
17     return x!=1;
18 }
19 bool miller_rabin(int n){
20     int s=(magic number size)
21     // iterate s times of witness on n
22     if(n<2) return 0;
23     if(!(n&1)) return n==2;
24     int u=n-1; int t=0;
25     // n-1 = u*2^t
26     while(!(u&1)) u>>=1, t++;
27     while(s--){
```

```

28     int a=magic[s]%n;
29     if(witness(a,n,u,t)) return 0;
30 }
31 return 1;
32 }

```

6.4 找一質因數

```

1 // does not work when n is prime O(n^(1/4))
2 int f(int x,int c,int mod){
3     return add(mul(x,x,mod),c,mod);
4 }
5 int pollard_rho(int n){
6     int c=1,x=0,y=0,p=2,q,t=0;
7     while(t++ % 128 || gcd(p,n)==1){
8         if(x==y) c++,y=f(x=2,c,n);
9         if(q=mul(p,abs(x-y),n)) p=q;
10        x=f(x, c, n);
11        y=f(f(y,c,n),c,n);
12    }
13    return gcd(p,n);
14 }

```

6.5 找全質因數

```

1 vector<int> ret;
2 void fact(int x){
3     if(miller_rabin(x)){
4         ret.push_back(x);
5         return;
6     }
7     int f=pollard_rho(x);
8     fact(f);fact(x/f);
9 }

```

7 數學

7.1 定理

- 錯排公式：(n 個人中，每個人皆不再原來位置的組合數)：
 $dp[0] = 1; dp[1] = 0;$
 $dp[i] = (i - 1) * (dp[i - 1] + dp[i - 2]);$
- 一個環上相鄰顏色不同的染色數量：
 $f(n, k) = (k - 1)^n + (-1)^n (k - 1)$
 n 是點的數量， k 是顏色的數量
- 最大流最小割原理：
 最大流量 = 最小割流量
 有向圖：S 邊 $level \neq -1$ T 邊 $level = -1$
 無向圖：只要一邊的 $level = -1$
 就是最小割邊
- König 定理：
 在二分圖中
 最大匹配 = 最小點覆蓋
 最小邊覆蓋 = 最大獨立集
 點數 = 最大匹配 + 最小邊覆蓋
 點數 = 最大獨立集 + 最小點覆蓋

A full-page sheet of white graph paper. The grid consists of thin, light gray horizontal and vertical lines forming small squares. There are 20 columns and 20 rows of these small squares, creating a large rectangular area for drawing or writing. The margins are uniform on all sides.